

Formation Engine Methodology

Section KF — Kinetic Formation, Organic Ontology, and the Thermodynamic–Kinetic Distinction

Formation Engine Development Team

Version 4.0.0 — April 2026

Contents

1	Introduction: What This Section Adds	1
2	The Central Distinction: Thermodynamic vs. Kinetic Product	2
3	Formation Likelihood: The Full Scoring Function	2
3.1	Thermodynamic Branch (S_{thermo})	2
3.2	Kinetic Branch (S_{kinetic})	2
3.3	Environmental Branch (S_{env})	3
3.4	Data Branch (S_{data})	3
3.5	Combined Formation Score	3
4	Universal Kinetic Engine Architecture	3
4.1	Why One Engine, Not Many	3
4.2	What Changes by Domain	3
4.3	Event Types	4
4.4	Rate Laws	4
5	Organic Ontology: A Separate Classification Layer	4
5.1	Why Organics Need Their Own Layer	4
5.2	Organic Family Tags	5
5.3	The Organic Identity Card	5
6	Integration with Existing Systems	6
6.1	Reaction Engine (§8–9)	6
6.2	Heat-Gated Control (§8b)	6
6.3	Gibbs Calculator	6
6.4	HGST Matrix	7
6.5	Report Engine	7
6.6	Visualization	7
7	Engine Lifecycle: Step-by-Step	7

8 Domain Adapter Examples	8
8.1 Organics	8
8.2 Crystal Growth	8
8.3 Droplet / Beam Transport	8
8.4 Decay-Aware Material	8
9 Static Structure as Special Case	9
10 File Manifest	9
11 Design Constraints	9

1 Introduction: What This Section Adds

Previous sections documented:

- §6 — formation physics (VSEPR prediction, MD evolution, FIRE quench, scoring)
- §8–9 — reaction prediction (Fukui, HSAB, Bell–Evans–Polanyi)
- §8b — heat-gated template control (the 3-digit h scalar)

This section introduces three additions that deepen the formation engine without breaking existing architecture:

1. **Kinetic formation scoring.** The distinction between *most stable product* and *most likely to actually form* becomes a first-class concept.
2. **Organic ontology layer.** A separate classification system for organic candidates, carrying chemistry-aware descriptors beyond what lattice-based categories provide.
3. **Universal kinetic engine.** One engine, many domain adapters. Not separate kinetic engines for organics, metals, droplets, crystals, beams, decay, and salts.

Implementation:

- `atomistic/kinetic/formation_kinetics.hpp`
- `atomistic/classify/organic_candidate.hpp`

Compatibility: integrates with the existing `ReactionEngine` (topology rewrites and Fukui scoring), `HeatGateController` (§8b), `GibbsCalculator` (thermodynamics.hpp), and `MaterialCase` / `TechnicalReport` (report_engine.hpp).

2 The Central Distinction: Thermodynamic vs. Kinetic Product

For any system—organic, inorganic, or mixed—two questions are fundamentally different:

Question	Meaning
What is most stable?	Lowest long-term free energy. The thermodynamic product. Reached given infinite equilibration time.
What actually forms?	Lowest effective barrier pathway. The kinetic product. Observed at finite reaction time.

The engine must compute *both* rankings and present them as first-class outputs. The expected output format is:

Candidate	Thermo Rank	Kinetic Rank	Expected Observation
A	1	3	likely after long equilibration
B	2	1	likely early product
C	3	2	transient intermediate

Implementation: FormationScore and CandidateRow in `formation_kinetics.hpp`.

3 Formation Likelihood: The Full Scoring Function

The probability that product i is observed is modelled as:

$$P(\text{product } i) \sim f(E_i, \Delta G_i, E_{a,i}, \tau, T, \text{solvent, crowding, path accessibility}) \quad (1)$$

This decomposes into four scoring branches.

3.1 Thermodynamic Branch (S_{thermo})

Based on the final static energy and Gibbs free energy of formation:

$$S_{\text{thermo}} = \sigma\left(\frac{-\Delta G_i}{kT}\right) \quad (2)$$

where σ is a sigmoid mapping $(-\infty, +\infty) \rightarrow [0, 1]$.

3.2 Kinetic Branch (S_{kinetic})

$$S_{\text{kinetic}} = w_1 A_{\text{path}} + w_2 R_{\text{encounter}} + w_3 S_{\text{steric}} + w_4 T_{\text{window}} + w_5 E_{\text{barrier}} \quad (3)$$

Component	Symbol	Meaning
Pathway accessibility	A_{path}	Is a viable reaction pathway geometrically open?
Encounter likelihood	$R_{\text{encounter}}$	Do reactants meet at sufficient rate?
Steric permissibility	S_{steric}	Is the transition sterically allowed?
Temperature suitability	T_{window}	Is the system in the right thermal window?
Barrier penalty	E_{barrier}	How high is the activation barrier?

Default weights: $w_1=0.20$, $w_2=0.20$, $w_3=0.20$, $w_4=0.15$, $w_5=0.25$.

Implementation: KineticWeights struct, FormationScore fields A.path through E.barrier_penalty.

3.3 Environmental Branch (S_{env})

$$S_{\text{env}} = \frac{1}{2} (S_{\text{solvent}} + S_{\text{crowding}}) \quad (4)$$

Solvent stabilisation and steric crowding from the local environment, evaluated per-event by the `DomainAdapter::compute_environment()` callback.

3.4 Data Branch (S_{data})

Incorporates meta-scores from the existing HGST and V4 scoring system (γ , Q_{data} , C_{compact}).

3.5 Combined Formation Score

$$S_{\text{form}} = \alpha S_{\text{thermo}} + \beta S_{\text{kinetic}} + \chi S_{\text{env}} + \delta S_{\text{data}} \quad (5)$$

Defaults: $\alpha=0.30$, $\beta=0.35$, $\chi=0.20$, $\delta=0.15$.

Implementation: `FormationKineticsEngine::score_candidate()`, `FormationKineticsEngine::rank_candidate()`

4 Universal Kinetic Engine Architecture

4.1 Why One Engine, Not Many

A separate kinetic engine for each domain (organics, metals, droplets, crystal growth, beam transport, decay, salt chemistry) would be a maintenance graveyard with extra folders.

At the highest level, kinetics is always:

State at time $t \rightarrow$ possible transitions $\rightarrow$ rates/probabilities $\rightarrow$ new state

The actual abstraction:

$$\mathcal{K} = \{ \mathcal{S}, \mathcal{E}, \mathcal{R}, \mathcal{U} \} \quad (6)$$

Symbol	Meaning
$\mathcal{S}$	State space: what exists right now
$\mathcal{E}$	Event space: what CAN happen
$\mathcal{R}$	Rate laws: how fast each event occurs
$\mathcal{U}$	Update rules: how the state changes after an event

4.2 What Changes by Domain

The engine is universal. The meaning of *state* and *barrier* changes.

Domain	State	Events	Barrier means
Organics	connectivity, conformer, medium, reactive site map	substitution, elimination, cyclisation, rearrangement, decomposition	orbital mismatch, steric hindrance, charge separation cost, torsional preorganisation
Crystal growth	lattice seed, coordination, defects, surface, temperature field	attachment, vacancy migration, dislocation, recrystallisation, defect annihilation	attachment penalty, diffusion activation, strain energy, surface energy mismatch
Droplet / beam	radius, temperature, velocity, oxidation, surface tension, gas density	flight, collision, evaporation, coalescence, impact, freezing	impact threshold, wetting threshold, thermal loss, contamination
Decay-aware material	isotope identity, decay age, coordination, daughter history, damage tally	$\alpha/\beta/\gamma$ emission, transmutation, lattice disruption, He accumulation	NOT just chemical barrier: hazard / deterministic emission law, damage propagation threshold

Implementation: DomainAdapter pure virtual base class. Each domain provides `enumerate_transitions()`, `rate_law_for()`, `apply_event()`, `compute_environment()`, `history_score()`.

4.3 Event Types

The `EventType` enumeration covers all domains without domain-specific subclassing. For events where the emission particle does not yet exist in the model (e.g. an alpha particle in a decay chain where the daughter nuclide has not been coded), the adapter **must** record the event and leave a placeholder. Silent data invention is forbidden.

Full list in `formation_kinetics.hpp`, lines covering `BOND_FORMATION` through `HELIUM_ACCUMULATION`.

4.4 Rate Laws

Type	Formula	Use
ARRHENIUS	$k = A e^{-E_a/k_B T}$	Chemical bond events, thermal processes
HAZARD	$\lambda = \ln 2/t_{1/2}$	Nuclear decay (temperature-independent)
COLLISION_THEORY	$k = \sigma v_{\text{rel}} n$	Gas-phase encounters, beam collisions
BARRIERLESS	$k = A$ (pre-exponential only)	Diffusion-limited reactions
DETERMINISTIC_STEP	fires at fixed Δt	MD-like stepping
CUSTOM	λ from adapter	Domain-specific extensions

5 Organic Ontology: A Separate Classification Layer

5.1 Why Organics Need Their Own Layer

The existing inorganic classification categories (`ElementCategory` in `comprehensive_elements.hpp`: noble metal, transition metal, alkali, actinide, halogen, etc.) are useful for materials-first logic.

For organics, these categories carry almost no information about chemical behaviour. Carbon is always NONMETAL, but the difference between an alkane and a nitrile is enormous.

5.2 Organic Family Tags

Family	Determines
ALKANE	saturated, low polarity, flexible chain
ALKENE	unsaturated, electrophilic addition sites
ALKYNE	linear geometry, strong triple bond
AROMATIC	rigid ring, delocalised π , electrophilic substitution
HETEROAROMATIC	aromatic with N/O/S, altered reactivity
ALCOHOL	OH donor, H-bond rich
ETHER	acceptor-only, low reactivity
KETONE	carbonyl, nucleophilic addition site
ALDEHYDE	terminal carbonyl, more reactive than ketone
CARBOXYLIC_ACID	acidic proton, H-bond donor+acceptor
ESTER	hydrolysis-prone, moderate polarity
AMIDE	resonance-stabilised, strong H-bond
AMINE	basic, nucleophilic, H-bond donor
NITRILE	linear, polarised triple bond
HALOGENATED_ORGANIC	leaving group, dipole, substitution-prone
ORGANOMETALLIC	metal-carbon bond, catalytic relevance
POLYMER_PRECURSOR	polymerisable double bond or functional group
CROSSLINKABLE	multi-functional, network-forming
RADICAL_PRONE	weak bonds, radical chain initiation
CONJUGATED	extended π , optical/electronic properties
RIGID_RING	limited conformational freedom
FLEXIBLE_CHAIN	many torsional DOF
DONOR_RICH	electron-rich, nucleophilic character
ACCEPTOR_RICH	electron-poor, electrophilic character

Implementation: OrganicFamily enum in `organic_candidate.hpp`.

5.3 The Organic Identity Card

Beyond lattice parameters and cohesive energy, the organic candidate object carries:

Descriptor block	Contents
Connectivity	formula, functional groups, ring count, aromatic count
Topology	sp3/sp2/sp counts, rotatable bonds, chiral centres
Chemistry scores	steric, polarity, H-bond donor/acceptor, conjugation, strain
Formation scores	thermo, kinetic, packing, decomposition risk, S_{form}
Reactive site map	per-atom Fukui indices, local softness, site type labels
Donor/acceptor map	per-atom H-bond donor/acceptor classification
Polarity map	per-atom partial charges and polarity contributions
Torsion profiles	per-rotatable-bond barrier heights and symmetry folds

Implementation: `OrganicCandidate` struct with nested `ReactiveSite`, `DonorAcceptor`, `PolaritySite`, `TorsionProfile` records. `OrganicClassifier` derives all fields from `atomistic::State`.

6 Integration with Existing Systems

6.1 Reaction Engine (§8–9)

The existing `ReactionEngine` handles:

- Fukui function computation
- HSAB site matching
- Bell–Evans–Polanyi barrier estimation
- Topology rewriting (bond break / form)

The kinetic formation engine sits one layer above: it decides *which* transitions are attempted, *at what rate*, and tracks time evolution. The reaction engine remains the workhorse for topology changes; the kinetic engine selects and sequences them.

6.2 Heat-Gated Control (§8b)

The heat parameter $h \in \{0, \dots, 999\}$ controls which bio-templates are active. The kinetic engine inherits this gating: events requiring inactive templates are excluded from the transition enumeration step.

6.3 Gibbs Calculator

The `GibbsCalculator` in `thermodynamics.hpp` provides ΔG values used directly by `FormationScore::delta_G_eV` and the thermodynamic branch scoring.

6.4 HGST Matrix

The 5D state vector $[\rho_D, \Gamma, S, \Pi, Q]$ feeds the data branch of the formation score via the existing `hgst::BondScorer` and `hgst::StateVector`.

6.5 Report Engine

`MaterialCase` and `TechnicalReport` gain a new optional field path: the kinetic ranking table can be included in reports alongside existing experiment results.

6.6 Visualization

The crystal card popup (`crystal_card_popup.py`) already exports JSON snapshots with γ , Q_{data} , C_{compact} . Future snapshots will include `thermo_rank`, `kinetic_rank`, and `S_form` from the kinetic engine.

The `atoms_vs_bonds.png` figure (Figure 1) illustrates the scaling relationship between atom count and bond count that underpins the transition enumeration: more bonds means more possible topology-change events.

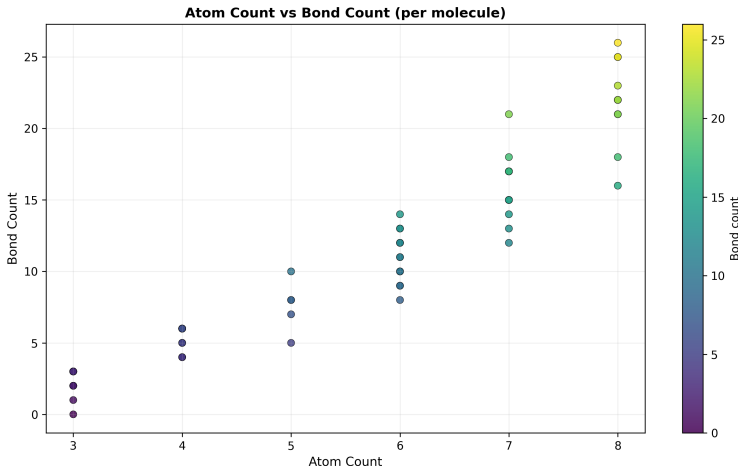


Figure 1: Atom count vs. bond count for representative structures. The bond count determines the upper bound on topology-change transitions enumerated per kinetic step.

7 Engine Lifecycle: Step-by-Step

A single kinetic step proceeds as:

1. **Enumerate.** `DomainAdapter::enumerate_transitions(state)` produces all transitions from the current state.
2. **Rate.** For each transition, the adapter provides a `RateLaw`; the engine evaluates $k_i = f(\text{state}, \text{env}, E_a, T)$.
3. **Score.** The engine computes all four branches (S_{thermo} , S_{kinetic} , S_{env} , S_{data}) and combines them into S_{form} .

4. **Select.** Rate-weighted stochastic selection (Gillespie-like) picks the next event.
5. **Apply.** `DomainAdapter::apply_event(state, event)` updates the state. Time advances by $\Delta t = -\ln(u)/\sum k_i$.
6. **Record.** The event is appended to the trail; lineage hash is updated.

Multi-step evolution runs steps 1–6 in a loop until `max_events` or `max_time` is reached.

8 Domain Adapter Examples

8.1 Organics

Adapter callback	Returns
<code>enumerate_transitions</code>	substitution, elimination, cyclisation, rearrangement, decomposition
<code>rate_law_for</code>	Arrhenius with BEP-estimated barriers
<code>apply_event</code>	topology rewrite via <code>ReactionEngine</code>
<code>compute_environment</code>	solvent dielectric screening, crowding from <code>steric_score</code>
<code>history_score</code>	formation path memory (radical chain propagation)

8.2 Crystal Growth

Adapter callback	Returns
<code>enumerate_transitions</code>	attachment, vacancy migration, dislocation, recrystallisation
<code>rate_law_for</code>	Arrhenius (diffusion barrier) or barrierless (adsorption)
<code>apply_event</code>	lattice topology update (add/remove atom, move defect)
<code>compute_environment</code>	local temperature gradient, surface energy anisotropy
<code>history_score</code>	defect accumulation penalty

8.3 Droplet / Beam Transport

Adapter callback	Returns
<code>enumerate_transitions</code>	flight, collision, evaporation, coalescence, impact, freezing
<code>rate_law_for</code>	collision theory ($k = \sigma v_{\text{rel}} n$)
<code>apply_event</code>	update radius, velocity, temperature, oxidation
<code>compute_environment</code>	local gas density, substrate proximity
<code>history_score</code>	prior collision chain (splashing history)

8.4 Decay-Aware Material

Adapter callback	Returns
<code>enumerate_transitions</code>	$\alpha/\beta/\gamma$ emission, transmutation, He accumulation
<code>rate_law_for</code>	Hazard: $\lambda = \ln 2/t_{1/2}$
<code>apply_event</code>	isotope transmutation, lattice disruption recording. If emission particle does not exist
<code>compute_environment</code>	radiation damage tally, daughter product neighbourhood
<code>history_score</code>	cumulative damage penalty, prior transmutation chain

9 Static Structure as Special Case

A “final observed structure” is one of three things:

1. **Lowest static energy** — the thermodynamic product. The result of FIRE quench to a local minimum on the PES.
2. **Sample-specific end energy** — the kinetic product. The state reached after a finite number of kinetic events at a given temperature.
3. **Energy at random time frame** — a snapshot of the time-evolving system, not necessarily at equilibrium.

The kinetic engine naturally handles all three: case 1 is `evolve()` with infinite time and zero temperature; case 2 is `evolve()` with finite `max.time`; case 3 is a single call to `step()` at an arbitrary point in the trajectory.

10 File Manifest

File	Purpose
<code>atomistic/kinetic/formation_kinetics.hpp</code>	Universal kinetic engine: <code>KineticState</code> , <code>TransitionEvent</code> , <code>EventType</code> , <code>RateLaw</code> , <code>FormationScore</code> , <code>KineticWeights</code> , <code>CandidateRow</code> , <code>DomainAdapter</code> , <code>FormationKineticsEngine</code>
<code>atomistic/classify/organic_candidate.hpp</code>	Organic ontology: <code>OrganicFamily</code> , <code>OrganicCandidate</code> (with nested site maps), <code>OrganicClassifier</code>
<code>atomistic/reaction/engine.hpp</code>	Existing reaction engine (unchanged, consumed by kinetic layer)
<code>atomistic/reaction/causal.hpp</code>	Causal discovery engine (unchanged, feeds history scoring)
<code>atomistic/reaction/heat_gate.hpp</code>	Heat-gated template control (unchanged)
<code>include/thermo/thermodynamics.hpp</code>	Gibbs calculator (unchanged, consumed for ΔG)
<code>include/hgst/hgst_matrix.hpp</code>	HGST scoring (unchanged, feeds data branch)
<code>include/core/report_engine.hpp</code>	Report engine (unchanged, receives ranking table)
<code>docs/figures/atoms_vs_bonds.png</code>	Scaling figure referenced in §6

11 Design Constraints

1. No data is hardcoded as a Z array. All descriptors are derived from the connectivity graph and element properties at runtime.
2. Terminology: use *atomistic* throughout. The word *meso* is forbidden.

3. If an emission particle does not yet exist in the model (e.g. alpha particle, positron), the adapter records the event and leaves a placeholder. It does not silently invent data.
4. Anti-black-box: every rate, every barrier, every event selection decision is inspectable and deterministic given seed.
5. The universal engine is ONE engine with domain adapters, not separate engines per domain.

VSEPR-SIM Formation Engine Methodology — Section KF: Kinetic Formation — Version 4.0.0, April 2026.